CCNA DevNet Lab Companion

> **What this is:** every lab in the **Lab Workbook**, runnable with zero hardware and zero cost on Cisco's free **DevNet Sandboxes** — cloud-hosted lab gear you book like a meeting room. The tasks, addressing, verification checklists, solutions and break-it challenges all stay in the Lab Workbook; this companion is the platform layer: which sandbox to book, how each topology translates, and what changes when the lab PC is Linux instead of VPCS.

> **When to use it:** your EVE-NG server is at home and you aren't — or you don't have one at all. Two tracks: the **full labs** on CML (browser, free Cisco account, and the VPN client — so, a machine you control), and an **At-Work Track** at the end of this book that needs nothing installed at all. Same labs, same commands, same muscle memory.

The Sandboxes

Two offerings at devnetsandbox.cisco.com matter for this book (a free Cisco/DevNet login gets you both):

Sandbox	Type	What it gives you	Used for
Cisco Modeling Labs (CML)	Reservable, ~4-hour sessions, VPN access	A full CML server: build any topology from Cisco's official images (IOL, IOL-L2, IOSv, IOSvL2, plus Linux hosts)	**Labs 1–17** — the EVE-NG stand-in (VPN client required → home machine)
IOS XE Always-On (Catalyst 8000v / 9000)	Shared, no reservation, no VPN	SSH and API access to one live IOS XE device	The zero-install **At-Work Track**, plus Chapter 22's REST APIs

Worth knowing about, but not needed here: **CML Free** — the same CML software as a free download you run locally, capped at 5 nodes. Labs 1–9 and 11–12 fit under that cap if you'd rather lab offline; the sandbox has no such squeeze.

Session Zero — Your First Reservation

The DevNet equivalent of the workbook's Lab 0: platform decisions and habits. Do this once before Lab 1.

Book it. Sign in at devnetsandbox.cisco.com, find the **Cisco Modeling Labs** sandbox, and reserve it. Spin-up takes 15–45 minutes; the confirmation email from `devnetsandbox@cisco.com` arrives when it's ready and contains your **VPN address and credentials**.

Connect. Install **Cisco Secure Client** (AnyConnect — the email links to a download if you don't have it), connect to the VPN address from the email, then open the CML web UI at the address shown on your reservation page and log in with the credentials shown there too. You're now standing in front of a lab server with real IOS images loaded.

Clear the stage. The sandbox boots with a demo lab (Multi Platform Network) already running. **Stop it** — it's burning the CPU and memory your lab wants. Then create a new, empty lab of your own.

Habits that pay off all book long — the sandbox versions:

1. **The clock is real.** Sessions run about **4 hours**. Every lab in the workbook is sized 30–60 minutes, so that's comfortable — but the capstone needs a multi-session plan (see Lab 17 below).
2. **The lab file is your save button.** When a lab works: `copy run start` on every device, **stop the nodes**, use CML's **Extract Configs** so the startup configs are written into the lab definition, then **download the lab** (a single `.yaml` file). Next session: import that file and every device boots exactly where you left it. This replaces EVE-NG's "Export all CFGs" habit — skip it and the reservation timer erases your evening.
3. **Size nodes before first boot.** IOL/IOL-L2 nodes get their interface count set when you drop them on the canvas — decide how many ports the lab needs first, because adding ports later means wiping the node. Two slots (e0/0–e1/3, eight ports) covers every switch in this book.
4. **Name everything, verify everything.** Unchanged from Lab 0 — `hostname SW1` first, a `show` command after every task.

Translating EVE-NG to CML

Node types

Workbook says	In CML pick	Notes
L2 switch (IOL L2 / viosl2)	**IOL-L2** (or IOSvL2)	IOL-L2 keeps the workbook's exact `e0/1`-style interface names
Router (IOL L3 / vIOS)	**IOL** (or IOSv)	See the interface-name note below
VPCS end host	**Alpine Linux**	A tiny real Linux box — see the survival kit

Everything the workbook configures — port security, DTP, PVST+, LACP, HSRP, DHCP snooping, DAI — works on IOL/IOL-L2 exactly as written, including quirks the solutions already flag (like `switchport trunk encapsulation dot1q`).

Interface names. Switch labs transfer verbatim: IOL-L2 has the same `e0/1`, `e2/0` names the workbook uses. Router labs are written with vIOS names (`g0/0`, `g0/1`); on an IOL router those become `e0/0`, `e0/1` — same positions, different letter (subinterfaces too: `g0/0.10` → `e0/0.10`). If renaming on the fly annoys you, use **IOSv** routers instead and keep the `Gi0/0` names at the cost of slower boots.

The Alpine survival kit

Alpine replaces VPCS as the lab PC. It's a real Linux host, which costs you four commands of learning and pays you back with real SSH, real DHCP leases and real listening sockets. The VPCS kit translates as:

```
$ sudo ip addr add 192.168.10.20/24 dev eth0      # ip <addr> <mask>
$ sudo ip link set eth0 up
$ sudo ip route add default via 192.168.10.1      # the gateway
$ ip addr show eth0                               # show ip
$ ip route                                        # ...and the gateway
$ ping 192.168.10.1                               # ctrl-c stops it
$ traceroute 192.168.10.1                         # trace
$ sudo udhcpc -i eth0                             # ip dhcp
$ ip neigh                                        # show arp
$ ip -6 addr show eth0                            # the SLAAC address
```

Two behavioral differences from VPCS: Linux `ping` runs **continuously** until ctrl-c (the workbook's "start a continuous ping" tasks get easier), and there is no `save` — an Alpine node's config vanishes on reboot, which is fine because the lab-download habit from Session Zero is the real save.

SSH from Alpine to a lab device may need legacy-crypto flags, because the IOS images speak older algorithms than a current OpenSSH offers by default:

```
$ ssh -o KexAlgorithms=+diffie-hellman-group14-sha1 \
     -o HostKeyAlgorithms=+ssh-rsa admin@192.168.1.2
```

Cabling

Links are drawn in the CML canvas just like EVE-NG. If the UI refuses to add or remove a link while a node runs (the workbook's "move the cable" moments in Labs 2 and 6), stop the two endpoint nodes, re-cable, start them again — IOL nodes boot in seconds, so this costs nothing.

The Labs, One by One

Node shopping lists and the (few) places where the platform changes the experience. Everything else — read the workbook and type.

Lab 1 — IOS Basics & SSH

Nodes: 1× IOL-L2, 1× Alpine.

The one genuine upgrade: task 5's "VPCS can't SSH" apology no longer applies. After configuring SSH on SW1, actually log in from PC1 with the legacy-crypto ssh command above — the full loop the workbook wished it could close. `reload` works normally in a CML console.

Lab 2 — Port Security

Nodes: 1× IOL-L2, 2× Alpine.

The attack step ("disconnect PC1, connect PC2 to e0/1") is a re-cable: stop both PCs, swap the link to e0/1, start PC2. Sticky learning, err-disable and recovery behave exactly as the solution describes.

Lab 3 — VLANs & Access Ports

Nodes: 1× IOL-L2 (two slots — the lab uses e1/0), 4× Alpine.

Lab 4 — Trunks, DTP & the Native VLAN

Nodes: 2× IOL-L2 (two slots each), 6× Alpine.

`%CDP-4-NATIVE_VLAN_MISMATCH` shows up on IOL-L2 just as promised.

Lab 5 — Inter-VLAN Routing, Two Ways

Nodes: Lab 4's, plus 1× IOL router.

Router-on-a-stick on IOL: `e0/0`, subinterfaces `e0/0.10` and `e0/0.20`. Part B's `ip routing` on the IOL-L2 works as written — it's a proper L3 switch.

Lab 6 — Spanning Tree

Nodes: 3× IOL-L2 (two slots each), 2× Alpine.

Check `show spanning-tree summary` before task 4: if the image defaults to legacy PVST+, you'll experience the ~30-second convergence the guide's 9.11 complains about — educational once, then `spanning-tree mode rapid-pvst` on all three switches to see the modern blink. Task 6's cable move is a stop/re-cable/start.

Lab 7 — EtherChannel

Nodes: 2× IOL-L2 (two slots each), 2× Alpine. Three parallel links draw fine in the canvas.

Lab 8 — Addressing Design (VLSM)

Nodes: 2× IOL routers, 4× Alpine. Still mostly paper — the sandbox can't help you carve a /22, which is the point.

Lab 9 — Static, Default & Floating Routes

Nodes: 3× IOL routers. Loopbacks work as written; remember `g0/0` → `e0/0`.

Lab 10 — OSPF Single Area

Nodes: same three routers — import last session's Lab 9 yaml and delete the statics, exactly as the workbook intends.

Lab 11 — IPv6 & SLAAC

Nodes: 2× IOL routers, 2× Alpine.

Task 3 changes flavor: there is no `ip auto` — Linux does SLAAC by itself the moment R1's router advertisement arrives. Just `ip -6 addr show eth0` and decompose what appeared. You may find **two** global addresses: the EUI-64 one (spot the `ff:fe` in the middle — that's the exam-relevant construction from 13.5) and possibly a randomized privacy address, which is worth meeting once: it's why real hosts don't leak their MAC to every web server.

Lab 12 — HSRP

Nodes: 3× IOL routers, 1× IOL-L2, 1× Alpine.

PC1's ARP table is `ip neigh` — the virtual MAC `0000.0c07.ac01` shows up there for the decoding task, and stays put across the failover just as promised.

Lab 13 — IP Services: DHCP, NTP, Syslog, SNMP

Nodes: Lab 12's, plus 2× Alpine.

`sudo udhcpc -i eth0` is the DORA trigger. And another upgrade: the workbook's syslog task shrugs "nothing will listen" — Alpine can. On PC2:

```
$ nc -lu -p 514
```

then make R1 log something (`shutdown`/`no shutdown` an unused interface) and watch the actual syslog message land on your screen. Seeing one real message makes 17.5's levels table permanent.

Lab 14 — NAT & PAT

Nodes: 2× IOL routers, 2× Alpine.

Third upgrade: the static-NAT port forward becomes testable. On PC1:

```
$ mkdir /tmp/www && echo hello-from-inside > /tmp/www/index.html
$ sudo busybox httpd -f -p 80 -h /tmp/www &
```

then from R2 — the "ISP", which has no route to the inside — prove the forward works: `telnet 203.0.113.1 80`, type `GET /` and watch a private-side web page answer through the translation.

Lab 15 — ACLs

Nodes: Lab 14's, plus 1× Alpine.

The vty task also gets real: SSH to R1 from PC1 (allowed) and from PC3 (refused) instead of taking `access-class` on faith.

Lab 16 — Layer 2 Defenses

Nodes: 1× IOL-L2 (two slots), 2× Alpine, 2× IOL routers (R1 legitimate, R4 rogue).

DHCP snooping and DAI behave as written, including the `no ip dhcp snooping information option` gotcha. Keep the workbook's DAI simulation as-is — Alpine ships no ARP-spoofing tools, and that's not an accident.

Lab 17 — Capstone

Nodes: roughly a dozen — 3× IOL-L2, 4× IOL routers, 3–4× Alpine. IOL images are tiny, so the sandbox carries this comfortably if the demo lab is stopped.

The 4-hour sessions force what the workbook politely suggests: staging. A working split is —

Session	Build	End-of-session yaml contains
1	HQ layer 2: VLANs, trunks, EtherChannel, STP roots, PortFast/BPDU Guard	a solid HQ switching fabric
2	Addressing + routing: VLSM plan (on paper before the reservation), OSPF, default route, HSRP	end-to-end reachability

Session	Build	End-of-session yaml contains
3	Services + security: DHCP/relay, NTP, syslog, PAT + port-forward, ACLs, snooping/DAI	the finished company
4	Break-and-fix finale, one sabotage at a time	your exam readiness

Extract configs and download the lab at the end of **every** session — the capstone yaml is the closest thing this companion has to a graduation certificate.

The At-Work Track — Zero Installs

Honesty first: the CML track above needs the Secure Client VPN **installed**, so it belongs to a machine you control. This track is for the other machine — the locked-down work laptop — and it needs nothing added to it:

The portal runs in the browser. Sign in at devnetsandbox.cisco.com, launch the **IOS XE Always-On** sandbox (Catalyst 8000v or 9000), and the page prints a hostname plus freshly generated credentials. No reservation, no VPN.

The SSH client is already on the machine. Windows 10/11 ships OpenSSH — open PowerShell (no admin rights needed):

```
PS> ssh <user>@<sandbox-host>
```

macOS: the same command in Terminal. That's the entire toolchain.

If the office firewall blocks outbound SSH, the browser itself is the fallback: paste `https://<sandbox-host>/restconf/data/ietf-interfaces:interfaces` into the address bar, sign in with the sandbox credentials at the prompt, and the interface table comes back as structured data right in the tab. (Which ports each Always-On box exposes varies — the catalog page lists them, and HTTPS has been switched off on some boxes in the past. SSH is the primary door.)

The shared-device rules, before drilling: it's one live router shared with strangers. Don't change credentials, don't erase or reload, don't save anything private into a config, expect it to be reset underneath you — and expect other people's half-finished configs all over it. Which, it turns out, is the training value.

Lunch-Break Drills

One shared device can't replace the workbook's topologies — but a surprising share of the exam is reading one router well. A rotation, one drill per sitting, each 10–20 minutes:

1. **Modes & help** (Lab 1, task 1) — user EXEC → privileged EXEC, `?` at every level, command abbreviation, Tab completion, `show` filtering with `| include` / `| section`. Look, don't touch: stay out of config mode unless the sandbox page says changes are welcome.
2. **Interface autopsy** (Ch 6, 23) — `show ip interface brief`, pick one interface, then `show interfaces <it>` and narrate every line: state, duplex, speed, drops, CRC. The exam's troubleshooting items are exactly this reading skill.
3. **Routing-table reading** (Ch 15) — `show ip route`: name every code letter, read each route's [AD/metric] pair, pick an address and do the longest-prefix match by hand, then check yourself with `show ip route <address>`.
4. **The crime scene** (Ch 23) — `show running-config` and reverse-engineer what previous visitors were building. Every visit the box is in a different state; explaining an unfamiliar config out loud is the troubleshooting section in miniature, and no home lab gives you this — your own configs are never surprising.
5. **Version & filesystem** (Ch 21) — `show version`: uptime, image name, config register; then `dir flash:`. Decode all of it from memory.
6. **Config kata** (any lab) — type a lab's full solution from memory into Notepad, then verify each command's syntax against the live device with `?` — without applying anything. Muscle memory, zero shared-state harm.
7. **RESTCONF** (Ch 22) — `curl.exe` also ships with Windows 10/11, so even the API call needs no install:

```
PS> curl.exe -k -u <user>:<password> `
    -H "Accept: application/yang-data+json" `
    https://<sandbox-host>/restconf/data/ietf-interfaces:interfaces
```

A real RESTCONF call returning real JSON — 22.4's diagram, live from a work laptop. Optional for the exam; quietly impressive in interviews.

Five of these a week keeps the CLI warm between CML sessions. The topologies — VLANs, OSPF adjacencies, HSRP failovers — still wait for the machine that can run the VPN client.

What Still Isn't Covered

The same two gaps as the workbook's closing note, one of them now half-closed:

Wireless (WLC GUI): still Packet Tracer's job — no sandbox gives you the WLC screens the exam screenshots.

Automation APIs: covered above — the Always-On sandbox is the answer the workbook pointed at.

Everything else: book a CML session, import yesterday's yaml, and keep typing.

— End of DevNet Lab Companion —